

AI Assistant Platform (AAP)

MVP Brief & Implementation Plan

1. Executive Summary

We are building an **AI Assistant Platform (AAP)** designed to augment team productivity across multiple roles such as Project Manager, Business Analyst, QA, Architect, and Developer. The platform uses **Model Context Protocol (MCP)** to dynamically retrieve and inject real-time, role-specific context from our existing tech stack: AWS, Atlassian (JIRA, Confluence), Slack, GitHub, and Google Suite.

2. MVP Goals

- Enable Claude chat UI to support **role-based agent behaviors** (PM, BA, QA, etc.)
- Allow agents to **retrieve real-time context** from integrated tools via MCP
- Design a **modular, extensible architecture** that supports rapid addition of new roles and sources

3. Core Architecture

Component	Description
Claude Desktop UI	Primary user interface for Phases 1-2; manages connections to both company MCP servers (via API Gateway + Cognito) and native MCP servers (direct OAuth). Replaced by Custom Agent UI in Phase 3
Custom Agent UI	Role-based web application hosted on ECS with real-time communication to n8n agents. Provides branded, enterprise-grade interface replacing Claude Desktop dependency
n8n Automation Platform	Self-hosted enterprise workflow automation platform serving as intelligent agent host with trigger processing, MCP integration, and complex workflow orchestration capabilities
AgentCore Runtime Agent	Intelligent agent running within Bedrock Runtime
n8n MCPs	MCP servers hosted and managed within n8n platform using native MCP Server Trigger nodes, providing workflow-based tool exposure with n8n's visual workflow management

n8n Triggers	Event-driven triggers within n8n platform processing webhooks, scheduled tasks, and service events to automatically initiate agent workflows and business process automation
Confluence Prompt Repository	Centralized prompt storage in Confluence pages managed by @Sylvain, providing immediate role-specific prompt access with version control through page history
Company MCP Servers	Custom AWS Lambda-based MCP servers that authenticate through API Gateway + Amazon Cognito federated with Google Workspace SSO for secure company data access
Native MCP Servers	Third-party maintained MCP servers (Atlassian, Google, Slack) with direct OAuth2 authentication to service providers, integrated across all phases
Amazon Cognito	Federated identity provider that integrates Google Workspace SSO with API Gateway for company MCP server authentication, providing JWT token management and RBAC
Google Workspace	Company's authoritative identity provider, integrated with Cognito for centralized SSO authentication and serving as the single source of truth for user identity
Django Admin Panel	Web-based management interface for prompt creation, editing, user role assignment, analytics dashboards, and content versioning with audit capabilities
Prompts (Company MCPs)	Evolution from static Confluence pages (Phase 1) to dynamic Lambda-hosted templates (Phase 2) to n8n workflow-integrated prompts (Phase 3) with company-specific validation and context injection
Tools (Company MCPs)	Custom business logic tools for internal systems including database queries, BI analytics, company GitHub operations, and validation workflows with enterprise authentication
Tools (Native MCPs)	Standard service operations provided by third-party MCP servers including ticket management, document operations, and communication tools with direct OAuth2 authentication
Datasources	Internal company data sources including databases, S3 buckets, and internal APIs accessed by company MCP servers with role-based access control and audit logging
Service Providers	External service APIs accessed by both company MCPs (with custom logic and enhanced security) and native MCPs (with standard operations and direct authentication)

4. Model Context Protocol (MCP) Architecture & Philosophy

4.1. Core Philosophy: Service-Centric Modularity

Our MCP implementation follows a **service-centric architecture** where each MCP server represents a single external service boundary. This approach aligns with MCP's core principle of creating focused, independent, and reusable context providers rather than monolithic orchestration layers.

Why Service-Based Split:

- **Single Responsibility:** Each MCP server owns exactly one external service domain (GitHub, JIRA, Confluence, Slack, Internal Database)
- **Independent Evolution:** Services can be updated, scaled, or replaced without affecting other components
- **Clean Authentication Boundaries:** Each MCP manages its own OAuth flows and service-specific credentials
- **Reusable Across Roles:** Any persona (Developer, PM, QA) can leverage any service MCP as needed
- **True Modularity:** New services can be added without modifying existing implementations

4.2. MCP Server Composition: Prompts + Tools

Each service-based MCP server contains two essential components working in tandem:

Prompt Design & Natural Validation Flow

Each MCP server exposes prompts using standard MCP JSON schema format. Here's the typical structure:

```
1. {
2.   "name": "code_review_analysis",
3.   "description": "Analyze pull requests for code quality, security, and compliance with team standards. Requires PR identification and project context for thorough analysis.",
4.   "arguments": [
5.     {
6.       "name": "pr_number",
7.       "description": "Pull request number or identifier to review",
8.       "required": true
9.     },
10.    {
11.      "name": "project",
12.      "description": "Project or repository name (e.g., 'payment-service', 'user-auth')",
13.      "required": true
14.    },
15.    {
16.      "name": "focus_areas",
17.      "description": "Specific areas to emphasize: security, performance, style, testing, documentation",
18.      "required": false
19.    }
20.  ]
21. }
```

How Required Fields Drive Natural Validation

When prompts specify **required fields**, Claude automatically understands what information is essential and naturally guides users to provide missing data through conversational flow.

Benefits of Required Fields:

- **Automatic Validation:** Claude recognizes when essential data is missing
- **Natural Questioning:** Claude asks for missing information conversationally
- **Quality Assurance:** Ensures tools receive sufficient context for meaningful responses
- **User Guidance:** Helps users understand what information produces better results

Validation Flow Example: Code Review Request

User Request: "Can you review this code?"

- **Step 1: Claude Recognizes Missing Required Data**

Claude sees the `code_review_analysis` prompt requires `pr_number` and `project`. Since these are missing, Claude **does not call the MCP tool** and instead responds:

"I'd be happy to review code for you. To provide a thorough analysis, I need: Which PR number should I review? & What project is this for?"

- **Step 2: User Provides Partial Information**

"It's PR #347"

- **Step 3: Claude Continues Data Gathering**

Claude recognizes `project` is still missing:

"Got it, PR #347. Which project is this for? (For example: payment-service, user-auth, etc.)"

- **Step 4: User Completes Required Data**

"payment-service project"

- **Step 5: Claude Calls MCP Tool with Complete Data**

Now Claude has all required fields and calls the GitHub MCP:

```
1. {  
2.   "pr_number": "347",  
3.   "project": "payment-service",  
4.   "focus_areas": null  
5. }
```

- **Step 6: MCP Tool Execution**

The GitHub MCP tool receives validated, complete data and can:

- Fetch PR #347 from payment-service repository
- Load project-specific coding standards
- Analyze code with proper context
- Return comprehensive review results

- **Step 7: Claude Delivers Contextual Response**

*"I've analyzed PR #347 for the payment-service project. Here's my review: **Security:** Payment processing follows secure patterns, but consider... **Performance:** API calls are optimized, however... **Standards:** Code adheres to payment-service guidelines except..."*

Overall assessment: Ready for merge after addressing the validation issue in PaymentProcessor.js line 142."

Contrast: Optional Fields Enable Enhancement

Optional fields like "Focus_area" allow users to **enhance** the analysis without blocking the process:

- Without Optional Field: "Review PR #347 in payment-service" → Proceeds with general review
- With Optional Field: "Review PR #347 in payment-service, focus on security" → Proceeds with security-focused review

4.3. Key Advantages of This Approach

Natural User Experience

- No technical error messages or validation failures
- Conversational flow feels like talking to a knowledgeable colleague
- Users learn what information improves results

Quality Assurance

- Tools always receive sufficient context
- Prevents poor results from incomplete data
- Ensures meaningful, actionable responses

Flexibility with Structure

- Required fields ensure minimum viable input
- Optional fields allow customization without complexity
- Claude adapts questioning style to user context

Progressive Enhancement

- Minimal: "Review PR #347 in payment-service"
- Enhanced: "Review PR #347 in payment-service focusing on security and performance"
- Expert: "Review PR #347 in payment-service focusing on PCI compliance and payment flow optimization"

Best Practices for Prompt Design

Required Fields Should Be:

- **Truly Essential:** Only mark fields as required if tools cannot function without them
- **User-Knowable:** Information users can reasonably be expected to provide
- **Specific:** Clear about what format/type of data is needed

Optional Fields Should:

- **Add Value:** Enhance results when provided
- **Have Sensible Defaults:** Tools should work well even when omitted
- **Enable Expertise:** Allow power users to customize behavior

Descriptions Should:

- **Guide Users:** Explain what information to provide and why
- **Give Examples:** Show format or typical values
- **Indicate Impact:** Explain how this field improves results

5. Tools: Service Operation Executors

Amazon Bedrock AgentCore is AWS's enterprise-grade managed service for deploying and operating AI agents at scale. It provides:

- **AgentCore Runtime:** Secure, serverless runtime with complete session isolation
- **AgentCore Identity:** Managed OAuth2 token vault and authentication service
- **AgentCore Gateway:** API-to-MCP conversion service
- **Native MCP Support:** Direct deployment and execution of MCP servers

AgentCore Runtime

- **Role:** Primary agent host and MCP server runtime environment
- **Capabilities:**
 - Native MCP server deployment and execution
 - Complete session isolation per user
 - Serverless scaling with 8-hour long-running support
 - Built-in authentication integration
- **Deployment:** AWS-managed service (no infrastructure management)

AgentCore Identity

- **Role:** Centralized OAuth2 token management for external services
- **Capabilities:**
 - Secure token vault for user-consented OAuth2 credentials
 - Automatic token refresh and lifecycle management
 - Integration with external identity providers
 - Per-user, per-service token isolation
- **Integration:** Native component of AgentCore Runtime

6. Tools: Service Operation Executors

Our architecture employs **three distinct types of MCP servers**, each with different implementation approaches, authentication mechanisms, and hosting strategies across the 3-phase delivery:

6.1. Native MCP Servers (Third-Party Providers)

These are **official MCP servers** maintained by service providers, offering robust, standardized functionality:

- **Built-in Authentication:** Each native MCP handles its own OAuth2 flows and credential management
- **Vendor Maintenance:** Service providers ensure compatibility and feature updates
- **Standard Operations:** Common tools for read/write operations within each service
- **Proven Reliability:** Battle-tested implementations with established error handling

- **Active Phases:** Available across all phases (1, 2, 3)

6.2. Company MCP Servers (AWS Lambda + API Gateway + Cognito)

These are **internally developed MCP servers** hosted on AWS Lambda, providing company-specific functionality with enterprise authentication:

- **Company SSO Integration:** Authentication managed through API Gateway + Amazon Cognito federated with Google Workspace
- **Custom Business Logic:** Company-specific validation rules, data enrichment, and workflow automation
- **Enhanced Prompts:** Role-specific prompt templates with context injection and validation
- **Internal System Access:** Direct integration with company databases, APIs, and proprietary tools
- **Data Protection:** Mandatory authentication layer for secure company data access
- **Active Phases:** Delivery 2 and 3

6.3. n8n MCP Servers (Self-Hosted with Embedded OAuth2)

These are **workflow-based MCP servers** hosted within the n8n platform, providing visual workflow automation with enterprise capabilities and **container orchestration** for complex processing tasks:

Core Capabilities

- **MCP Server Provider:** Exposes complex workflows as discoverable MCP tools via MCP Server Trigger nodes
- **Embedded OAuth2:** n8n manages OAuth2 flows for external services within workflow nodes, with AgentCore Identity integration for user-specific token retrieval
- **Visual Workflow Design:** MCP tools created through n8n's visual workflow builder with drag-and-drop orchestration
- **MCP Server Trigger Nodes:** Native n8n nodes that expose workflows as MCP tools with standardized discovery and execution
- **Synchronous Result Return:** Workflows maintain standard MCP request-response patterns by waiting for all processing steps (including containerized tasks) to complete before returning formatted results to the requesting agent

Enterprise Orchestration & Container Management

- **ECS Task Orchestration:** Workflows can programmatically spawn dedicated ECS Fargate tasks for resource-intensive operations (testing, data processing, AI model inference)
- **Multi-Container Coordination:** Single workflow can coordinate multiple containerized services with different resource requirements and execution environments

- **AWS Service Integration:** Direct integration with ECS RunTask API, Lambda functions, S3, and other AWS services through native n8n nodes
- **Resource Isolation:** Dedicated container tasks ensure processing isolation and prevent resource contention with main n8n service
- **Task Completion Monitoring:** Workflows actively monitor containerized task status and wait for completion before proceeding with result processing

Advanced Automation Patterns

- **Trigger-Based Automation:** Event-driven workflows from webhooks, schedules, and service events with automatic container provisioning
- **Dynamic Resource Allocation:** Workflows can adjust container specifications (CPU, memory, timeout) based on workload requirements
- **Result Aggregation:** Collect and process results from multiple parallel container executions within single workflow, formatting them for agent consumption
- **Error Handling & Retry Logic:** Built-in failure management with container restart capabilities, timeout handling, and exponential backoff
- **Synchronous Execution Flow:** Despite internal complexity, workflows present simple synchronous interface to calling agents with complete results or structured error responses

Technical Implementation

- **Self-Hosted Control:** AWS ECS Fargate hosting with complete control over workflow logic, data processing, and container orchestration
- **Container Registry Integration:** Direct integration with Amazon ECR for custom container image management
- **Networking & Security:** VPC-based networking with security group controls for container-to-container and container-to-external service communication
- **Monitoring & Observability:** Built-in execution logging with CloudWatch integration for container lifecycle management
- **Result Storage & Retrieval:** Temporary storage mechanisms (S3, EFS) for intermediate results with automatic cleanup and result formatting for agent responses

Use Case Example: QA Testing Automation

None

QA Agent → n8n MCP → Workflow Steps:

└─ Data Processing (n8n nodes)

└─ ECS RunTask → Cypress Container → API Testing

└─ Task Monitoring & Completion Wait

└─ Result Collection (S3/EFS)

└─ Result Formatting & Validation

└─ Formatted Results → Agent Response

Container Task Configuration:

- **Cypress Testing:** Dedicated Fargate tasks (1-4 vCPU, 2-8GB RAM) with full browser capabilities and result persistence
- **Data Processing:** Custom containers for Excel analysis, report generation, or ML model execution with structured output
- **Integration Testing:** Multi-service test orchestration with isolated network environments and comprehensive result aggregation

Result Flow Management:

- **Workflow Blocking:** n8n workflows pause execution during container task processing to maintain synchronous behavior
- **Timeout Handling:** Configurable timeouts with partial result recovery and structured error reporting
- **Result Transformation:** Container outputs automatically formatted into agent-friendly response structures with success indicators, metrics, and detailed findings

6.4. Unified Tool Execution Model

All three types handle service interactions through standardized MCP protocols:

- **API Integration:** Manages authenticated connections using appropriate auth mechanisms (OAuth2 for native/n8n, SSO for company)
- **Data Retrieval:** Fetches specific information based on clarified requirements from external services or internal systems
- **Operation Execution:** Performs validated actions (creating tickets, updating documentation, querying databases)
- **Response Formatting:** Structures returned data for optimal consumption following MCP specifications
- **Error Handling:** Manages service-specific error conditions and retry logic appropriate to each service type

6.5. Agent Architecture Evolution

Phases 1-2: Claude Desktop as Orchestrator

Claude Desktop serves as the intelligent orchestrating agent that seamlessly coordinates between native and company MCP servers:

- **Triple Architecture Awareness:** Manages native MCPs, company MCPs, and n8n MCPs seamlessly
- **Role Awareness:** Understands user context and role requirements (Developer, PM, QA, Architect) to select appropriate tools using the correct prompt on the company MCP Prompt
- **Multi-MCP Coordination:** Intelligently routes requests to appropriate MCP servers based on intent
- **Cross-Service Workflows:** Orchestrates complex operations spanning multiple services
- **Authentication Management:** Handles different authentication flows transparently
- **Context Synthesis:** Combines responses from multiple MCP servers into coherent, role-specific insights

Phase 3: AgentCore Runtime as Enterprise Agent Host

AgentCore Runtime replaces Claude Desktop as the enterprise-grade agent hosting platform:

- **Native MCP Runtime:** Directly hosts and executes Company MCP servers within AgentCore's secure runtime environment
- **Session Isolation:** Each user interaction runs in a dedicated MicroVM with complete CPU, memory, and filesystem isolation
- **Multi-MCP Orchestration:** Seamlessly coordinates between Company MCPs (internal), Native MCPs (external), and n8n MCPs (workflow automation)
- **Enterprise Authentication:** Built-in AgentCore Identity manages OAuth2 token vault and external service authentication
- **Serverless Scalability:** AWS-managed infrastructure supports thousands of concurrent users with automatic scaling
- **Long-Running Workloads:** Industry-leading 8-hour session support for complex, multi-step agent tasks
- **Custom UI Integration:** Direct integration with Angular-based enterprise interface via AgentCore APIs

6.6.Service Boundary Definitions

Company MCP Servers ("Lambda + API Gateway" or AgentCore Runtime + Cognito)

Company Prompts MCP Server

- **Deployment:** AWS Lambda behind API Gateway with Cognito authentication

- **Domain:** Request validation, context enrichment, and prompt template management. To be used **EVERYTIME** for all request, as it will host all the business knowledge (procedure & requirements)
- **Authentication:** Amazon Cognito federated with Google Workspace SSO
- **Tool Capabilities:** Input validation, context injection, template processing, approval workflows

Company Database MCP Server

- **Deployment:** AWS Lambda behind API Gateway with Cognito authentication
- **Domain:** Internal company database operations and business intelligence
- **Authentication:** Amazon Cognito federated with Google Workspace SSO
- **Tool Capabilities:** User queries, system metrics, project statistics, custom reporting & others

Company GitHub MCP Server *(Conditional - Phase 2)*

- **Deployment:** AWS Lambda behind API Gateway with Cognito authentication
- **Domain:** Company-specific GitHub operations with enhanced business logic
- **Authentication:** Amazon Cognito federated with Google Workspace SSO
- **Tool Capabilities:** PR analysis with company rules, code diff retrieval, branch management

Native MCP Servers (Third-Party Maintained)

Atlassian MCP Server

- **Deployment:** Native MCP with direct OAuth2 authentication to Atlassian Cloud
- **Domain:** Official JIRA, Confluence, and Atlassian suite operations
- **Authentication:** Native OAuth2 flow with Atlassian identity provider
- **Tool Capabilities:** Standard ticket operations, documentation management, project tracking

Google Workspace MCP Server

- **Deployment:** Native MCP with direct OAuth2 authentication to Google APIs
- **Domain:** Google Drive, Gmail, Calendar, and Workspace document operations
- **Authentication:** Native OAuth2 flow with Google identity provider
- **Tool Capabilities:** Document management, email operations, calendar scheduling and others

n8n MCP Servers (Self-Hosted with Embedded OAuth2)

n8n Slack MCP Server

- **Deployment:** n8n workflow using MCP Server Trigger nodes
- **Domain:** Standard Slack communication and collaboration features
- **Authentication:** Embedded OAuth2 managed by n8n GitHub credential nodes
- **Tool Capabilities:** Message retrieval, channel management, notification handling and others

n8n QA Testing MCP Server

- **Deployment:** n8n workflow using MCP Server Trigger nodes with ECS task orchestration
- **Domain:** Automated testing workflows for API validation and quality assurance
- **Authentication:** AgentCore Identity for user-specific test execution and result attribution
- **Tool Capabilities:** Cypress test execution, Excel data processing, test data generation, API validation, result aggregation and reporting

n8n Data Processing MCP Server

- **Deployment:** n8n workflow using MCP Server Trigger nodes with containerized processing
- **Domain:** ETL operations, document analysis, and business data transformation
- **Authentication:** AgentCore Identity for secure data access and user-specific processing
- **Tool Capabilities:** Excel/CSV/JSON transformation, data validation, format conversion, and business rule application

n8n others MCP (not native yet)

- **Deployment:** n8n workflow using MCP Server Trigger nodes
- **Domain:** To discovers
- **Authentication:** Embedded OAuth2 managed by n8n GitHub credential nodes
- **Tool Capabilities:** To discovers

6.7. Cross architecture coordination (Phase3)

Prompt-Driven Multi-MCP Orchestration

- **Authentication Entry Point:** All requests start with AgentCore Runtime user authentication via Cognito-validated JWT tokens
- **Unified Prompts Entry:** After authentication, Company Prompts MCP called regardless of target MCP type (Company Prompts MCP runs internally within AgentCore Runtime)

- **Business Logic Orchestration:** Prompts MCP defines which combination of MCP types required:
 - Company MCPs for internal data and validation (deployed within AgentCore Runtime)
 - Native MCPs for standard service operations (external, using AgentCore Identity tokens)
 - n8n MCPs for complex workflow automation (external, via MCP Server Trigger nodes)
- **Sequential Tool Execution:** AgentCore Runtime agent executes tools in prompt-defined order:
 - Each tool call uses validated data from prompt phase
 - Company MCPs access user context directly from AgentCore session
 - External MCPs receive OAuth2 tokens from AgentCore Identity vault
 - Results from each tool may trigger additional prompt validation
 - Process continues until prompt completion criteria satisfied
- **Multi-Type Synthesis:** AgentCore Runtime agent combines responses from all MCP types into coherent, role-specific insights while maintaining prompt-defined business logic and validation requirements, with complete session isolation ensuring user data security

6.8.Key Benefits of Prompt-First Architecture:

Business Logic Centralization

- **Single Source of Truth:** All business processes defined in Company Prompts MCP
- **Consistent Validation:** Same business rules applied across all MCP types
- **Audit Trail:** Complete tracking of business process compliance

Quality Assurance

- **Data Validation:** No tool execution without complete, validated data
- **Business Compliance:** Company rules enforced before any external service calls
- **Error Prevention:** Prompt validation prevents incomplete or incorrect operations

Workflow Orchestration

- **Process Definition:** Prompts define multi-step workflows and tool sequences
- **Conditional Logic:** Business rules determine which tools to call and when
- **Completion Criteria:** Clear definition of when tasks are fully completed

This prompt-first pattern ensures that **every operation begins with business logic validation** and **no tools are executed without proper data validation**, regardless of whether the final execution happens via Company MCPs, Native MCPs, or n8n MCPs.

7. Technological Stack

Layer	Technology	Purpose
MCP Framework	MCP SDK (Python) + n8n MCP Nodes	Core protocol implementation for building MCP servers and n8n workflow integration
Backend Language	Python 3.11+	Primary development language for Lambda MCPs and Django admin panel
Web Framework	FastAPI (Lambda & AgentCore) + Django (Admin Panel)	HTTP server framework for Lambda MCP servers, AgentCore, and web-based administration
Frontend Framework	Angular + TypeScript	Custom Agent UI for Phase 3 enterprise interface
Authentication	API Gateway + Amazon Cognito + Google Workspace SSO	Company SSO integration and enterprise access control
Automation Platform	n8n Enterprise (Self-Hosted)	Visual workflow automation and agent hosting for Phase 3
HTTP Client	httpx / requests	External service API calls across all MCP types
Database	PostgreSQL (Primary) + DynamoDB (Cache)	Prompt storage, user management, and state management
Content Management	Confluence (Phase 1) + Django Admin (Phase 2-3)	Prompt repository evolution from static to dynamic management
Storage	S3 (Static Assets) + PostgreSQL (Structured Data)	File storage and structured data management
Cloud Infrastructure	AWS Lambda + API Gateway (Company MCPs) + ECS Fargate (Custom UI)	Serverless MCP deployment and containerized UI hosting
Containerization	Docker (Lambda + ECS + n8n)	Consistent deployment across Lambda, custom UI, and n8n platform

Secrets Management	AWS Secrets Manager + n8n Credentials Store	API keys, service credentials, and OAuth tokens
Monitoring	CloudWatch + n8n Built-in Monitoring + Datadog (Optional)	Comprehensive logging, metrics, and performance monitoring
Logging	Python logging + CloudWatch + n8n Execution Logs	Debugging and audit trail across all platforms
Testing	pytest + httpx (Lambda) + n8n Test Workflows	Unit, integration, and workflow testing

8. Authentication, Authorization & Access Control Architecture

8.1. Authentication Overview

Our authentication system leverages **Google Workspace as the authoritative identity provider**, federated through **Amazon Cognito**, providing unified single sign-on across all platform components. This approach ensures **enterprise-grade security** while maintaining seamless user experience through standardized JWT token-based authentication.

The architecture supports **two distinct authentication flows**: inbound authentication for user access to the platform, and outbound authentication for agent access to external services, with **session isolation** and **centralized token management** ensuring secure multi-user operations.

8.2. Token Exchange & User Context

JWT Token Structure

The platform exchanges **Cognito-issued JWT tokens** containing essential user context:

json

```
JSON
{
  "sub": "user-uuid",
  "email": "john.doe@company.com",
  "cognito:groups": ["developers@company.com", "pm-team@company.com"],
  "custom:role": "senior-developer",
```



```
"exp": 1735689600,  
"iat": 1735603200  
}
```

User Context Propagation

- **Phase 2:** JWT validated at API Gateway, user context injected into Lambda event headers
- **Phase 3:** JWT validated by AgentCore Runtime, user context available to all internal Company MCPs via session context
- **External Services:** OAuth2 access tokens stored per-user in AgentCore Identity vault, retrieved on-demand

OAuth2 Token Management

External service access uses **user-specific OAuth2 tokens** managed by AgentCore Identity:

- **Consent Flow:** Per-user, per-service authorization with secure token storage
- **Token Lifecycle:** Automatic refresh with re-consent fallback on refresh failure
- **Scope Isolation:** Service-specific permissions maintained separately for each user

8.3. Access Control & Restrictions

Multi-Level Access Control Strategy

Level 1: Platform Access (SSO-Based)

Google Workspace group membership determines basic platform access:

- **Authentication Gate:** Only users in authorized Google Workspace groups can authenticate
- **Role Assignment:** Groups mapped to platform roles (developer, pm, qa, architect, admin)
- **Session Context:** User groups automatically available to all platform components

Level 2: Prompt-Level Restrictions (Primary Control Point)

Company Prompts MCP filters available prompts based on user context:

- **Prompt Visibility:** Role-based filtering of available prompts and workflows

- **Business Logic Gate:** Prompts define role-specific validation rules and data access patterns
- **Dynamic Discovery:** Users only see prompts they're authorized to execute

Level 3: Tool-Level Validation (Secondary Enforcement)

Individual Company MCPs enforce business logic permissions:

- **Operation Validation:** Tool execution validated against user groups and roles
- **Data Access Control:** Database queries and internal system access controlled per user
- **Audit Trail:** All operations logged with user context for compliance

Level 4: External Service Permissions (OAuth2-Based)

External services enforce their own permission models:

- **User-Specific Tokens:** Each user's OAuth2 tokens carry their external service permissions
- **Service-Level Authorization:** GitHub, Slack, JIRA permissions determined by user's account in those services
- **Consent Management:** Users explicitly authorize agent access to their external accounts

Restriction Inheritance Flow

None

Google Workspace Groups → Platform Roles → Available Prompts → Tool Operations → External Service Access

Each level inherits and refines restrictions from the previous level, ensuring **defense-in-depth** security while maintaining **user experience simplicity**.

8.4. Strategy by delivery phase

Phase 2: Lambda-Based Authentication Architecture

Core Components

API Gateway + Cognito + Google Workspace Architecture leverages AWS managed authentication at the API Gateway level while integrating with company SSO infrastructure.

Authentication Components:

- **API Gateway Authentication Layer:** Validates incoming requests and manages CORS
- **Amazon Cognito Integration:** Federated identity provider with Google Workspace, JWT token management
- **Google Workspace SSO Federation:** Authoritative identity source with SAML/OIDC integration

Phase 2 Authentication Flow

None

User → Google Workspace SSO → Cognito → JWT Token → API Gateway → Lambda MCP

Access Control Implementation:

- **JWT Validation:** API Gateway Cognito authorizer validates tokens before Lambda execution
- **User Context Injection:** Lambda receives user context via `event.requestContext.authorizer`
- **Permission Validation:** Each Lambda MCP validates user groups against requested operations

Phase 3: AgentCore-Based Authentication Architecture

Multi-Layered Authentication Model

Layer 1: User-to-Agent Authentication (Inbound)

None

User → Google Workspace SSO → Cognito → JWT Token → AgentCore Runtime

- **Cognito User Pool:** Federated with Google Workspace SSO (unchanged from Phase 2)
- **AgentCore Runtime:** Validates JWT tokens with Cognito discovery endpoint
- **Session Creation:** Each validated user gets isolated MicroVM environment

Layer 2: Agent-to-External Services Authentication (Outbound)

None

AgentCore Runtime → AgentCore Identity → User-Specific OAuth2 Tokens → External Services

- **Initial Consent:** Per-user, per-service authorization with secure token storage
- **Runtime Token Retrieval:** Agent requests user-specific tokens from vault
- **Automatic Token Refresh:** Lifecycle management with re-consent fallback

Access Control Evolution

Company MCP Authentication (Simplified):

- **Phase 2:**
 - User → API Gateway → Cognito Authorizer → Lambda MCP
- **Phase 3:**
 - User → AgentCore Runtime (with Company MCPs deployed internally)

Benefits:

- **Direct User Context:** Company MCPs access user context from AgentCore session
- **No API Gateway:** Eliminated external authentication layer for internal operations
- **Session Isolation:** Automatic user data separation via MicroVM architecture
- **Centralized OAuth2:** All external service tokens managed by AgentCore Identity

8.5. Key Differences Between Phases

Phase 2 Characteristics:

- API Gateway required for all Company MCP requests
- Manual JWT validation in Lambda code
- External service authentication managed separately
- User context passed via Lambda event headers

Phase 3 Characteristics:

- Company MCPs run directly in AgentCore Runtime
- Automatic session context with user permissions
- Centralized OAuth2 token management via AgentCore Identity
- Complete session isolation with MicroVM architecture

9. Authentication Strategy by Delivery phase

9.1.Phase 2: Lambda-Based Authentication Architecture

Core Components

API Gateway + Cognito + Google Workspace Architecture Our Phase 2 custom MCP servers leverage AWS managed authentication at the API Gateway level while integrating with company SSO infrastructure.

Authentication Components

API Gateway Authentication Layer

- **Primary Access Control:** Validates incoming requests from Claude Desktop
- **Request Routing:** Directs authenticated requests to appropriate Lambda MCP servers
- **Token Validation:** Integrates with Amazon Cognito for user identity verification
- **CORS Management:** Handles cross-origin requests from Claude Desktop clients

Amazon Cognito Integration

- **Identity Federation:** Configured as federated identity provider with Google Workspace
- **Token Management:** Issues and validates JWT tokens for authenticated users
- **User Pool Integration:** Maps Google Workspace users to Cognito identities
- **Session Management:** Handles token refresh and session lifecycle

Google Workspace SSO Federation

- **Identity Source:** Company's existing Google Workspace serves as authoritative identity provider
- **SAML/OIDC Integration:** Standard federation protocols with Google Workspace
- **User Attributes:** Syncs user information, groups, and permissions from Google Workspace
- **Centralized Management:** Leverages existing company identity and access management

Phase 2 Authentication Flow

1. Claude Desktop → API Gateway → Cognito → Lambda MCP → Company data

Runtime Authentication Steps:

- Claude Desktop → API Gateway (with JWT token headers)
- API Gateway → Amazon Cognito (token validation)
- Cognito → Google Workspace (if token refresh needed)
- Cognito validates user identity and permissions
- API Gateway → Lambda MCP (with user context)
- Lambda processes request with authenticated user context
- Response flows back through API Gateway to Claude Desktop

9.2.Phase 3: AgentCore-Based Authentication Architecture

Multi-Layered Authentication Model

Phase 3 introduces AgentCore Runtime with centralized OAuth2 token management, eliminating API Gateway complexity for Company MCPs while adding sophisticated external service authentication.

Layer 1: User-to-Agent Authentication (Inbound)

1. User → Google Workspace SSO → Cognito → JWT Token → AgentCore Runtime

Components:

- **Cognito User Pool:** Federated with Google Workspace SSO (unchanged from Phase 2)
- **AgentCore Runtime:** Validates JWT tokens with Cognito discovery endpoint
- **Session Creation:** Each validated user gets isolated MicroVM environment

Layer 2: Agent-to-External Services Authentication (Outbound)

1. AgentCore Runtime → AgentCore Identity → OAuth2 Tokens → External Services

OAuth2 Token Management Flow:

- **Initial Consent** (Per User, Per Service)
 - User grants permission: "Allow agent to access Slack"

- AgentCore Identity performs OAuth2 flow with Slack
- OAuth2 tokens stored in secure vault
- User consent recorded
- **Runtime Token Retrieval**
 - Agent needs Slack access for User John
 - Agent requests: "Get John's Slack token"
 - AgentCore Identity returns valid OAuth2 token
 - Agent uses token with Native MCP or n8n workflow
- **Automatic Token Refresh**
 - Token expiration detected
 - AgentCore Identity attempts refresh
 - If refresh fails: re-consent prompt to user
 - New tokens stored in vault

10. Enterprise Chat Interface

A **custom Angular-based chat interface** is required to replace Claude Desktop dependency and provide enterprise-grade user interaction with the AgentCore platform. The application will be **containerized and deployed on ECS Fargate** with auto-scaling capabilities.

10.1. Core Components

- **Chat Interface:** Real-time messaging with WebSocket connection to AgentCore Runtime
- **Session Management:** Persistent conversation history with user-specific storage
- **MCP Tool Discovery:** Dynamic prompt/tool browsing based on user permissions
- **OAuth2 Flow Handler:** Embedded consent management for external services

10.2. Tech Stack

- **Frontend:** Angular 17+ with TypeScript, Angular Material UI components
- **Authentication:** **Unified Cognito + Google Workspace SSO** (same as AgentCore Runtime)
- **Communication:** WebSocket for real-time messaging, HTTP for tool discovery
- **Deployment:** Docker container on ECS Fargate with Application Load Balancer
- **State Management:** NgRx for conversation history and user session state
- **PWA Support:** Service workers for offline capability and mobile responsiveness

10.3. Unified Authentication Flow

None

User → Angular UI → Google Workspace SSO → Cognito → JWT Token

Same JWT token used for:

- └─ Angular UI session management
- └─ AgentCore Runtime authentication

10.4. Integration Points

- AgentCore Runtime API for agent communication (using same JWT)
- Cognito for unified user authentication and session management
- OAuth2 consent redirects for external service authorization
- Company MCP endpoints for user permissions and available tools

This provides **single sign-on experience** where users authenticate once to access both the UI and all AgentCore capabilities, eliminating Claude Desktop dependency while maintaining enterprise security standards.

11. Implementation & Milestones

11.1. Delivery 1 (Phase 2)

Week 1: Foundation & Authentication

Focus: Get authentication working end-to-end

- Set up Cognito User Pool + Google Workspace federation
- Configure API Gateway with Cognito authorizer
- Develop company prompts
- Deploy containerized Lambda pipeline (SAM/CDK + Docker + ECR)
- Implement Company Prompts MCP with PostgreSQL schema
- End-to-end authentication validation

Week 2: Core Company MCPs

Focus: Essential business logic MCPs

- Deploy Database MCP Server (containerized Lambda)
- Enhanced Prompts MCP with dynamic templates and validation
- Multi-MCP workflow testing and performance optimization
- Container layer optimization across MCPs

Week 3: Admin Panel

Focus: Django management interface

- Django setup with Cognito authentication
- User management and role assignment interface
- Basic analytics dashboard and prompt performance tracking
- User group synchronization with Google Workspace

Week 4: n8n Integration & Production

Focus: n8n MCPs and deployment

- Self-hosted n8n deployment with MCP Server Trigger workflows
- Integration testing and multi-MCP workflow validation
- Production environment setup with monitoring and documentation

Delivery phase 2

- **Admin Dashboard:** Django management interface with PostgreSQL database and Redis cache
- **Company MCPs:** Database access and Prompts server, containerized and deployed as Lambda functions with API Gateway authentication
- **Authentication System:** Cognito + Google Workspace SSO federation with API Gateway integration
- **n8n Platform:** Self-hosted on ECS Fargate with MCP Server Triggers for Slack integration and QA testing automation
- **Native MCP Integration:** Basic external service connections (Atlassian, Google) for testing and validation

11.2. Delivery 2 (Phase 3)

Week 1: AgentCore Runtime Foundation

Focus: AgentCore deployment and initial MCP migration

- Deploy AgentCore Runtime environment with Cognito authentication
- Convert Company Prompts MCP to AgentCore Runtime format
- Remove API Gateway dependency and test user context access

Week 2: Complete Company MCP Migration

Focus: All Company MCPs in AgentCore Runtime

- Migrate Database MCP to AgentCore Runtime
- Update authentication to use AgentCore session context
- Multi-MCP workflow validation and performance testing

Week 3: AgentCore Identity + External Services

Focus: Security of AgentCore and UI

- Set up unified OAuth2 Security authentication between UI and AgentCore
- Configure AgentCore Identity OAuth2 token vault
- Set up consent flows and connect Native MCPs (Atlassian, Google, Slack)
- Test token lifecycle and user-specific token isolation

Week 4: Final Integration & Production

Focus: All Company MCPs in AgentCore Runtime

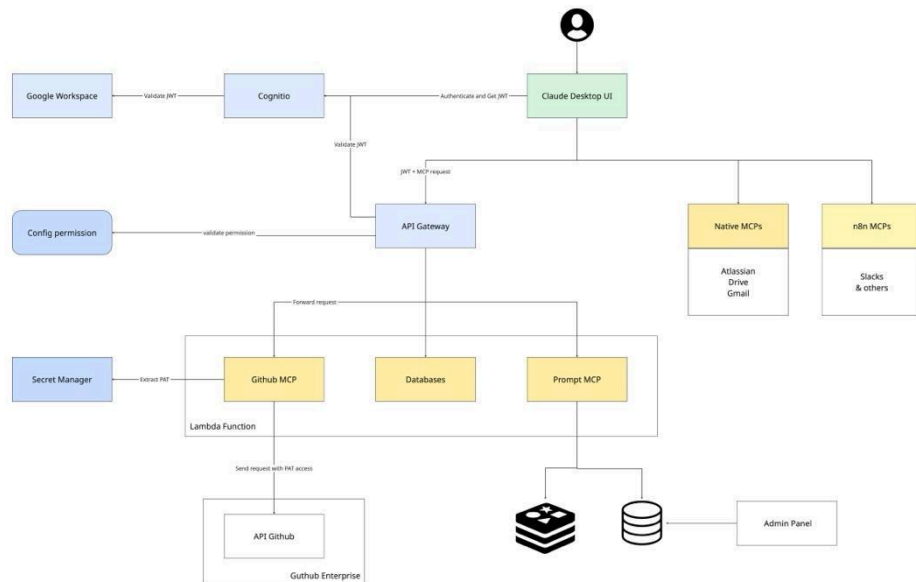
- Integrate AgentCore with UI
- Validate n8n MCP integration with AgentCore Runtime
- End-to-end user scenarios and security testing
- Production deployment with comprehensive testing

Delivery phase 3

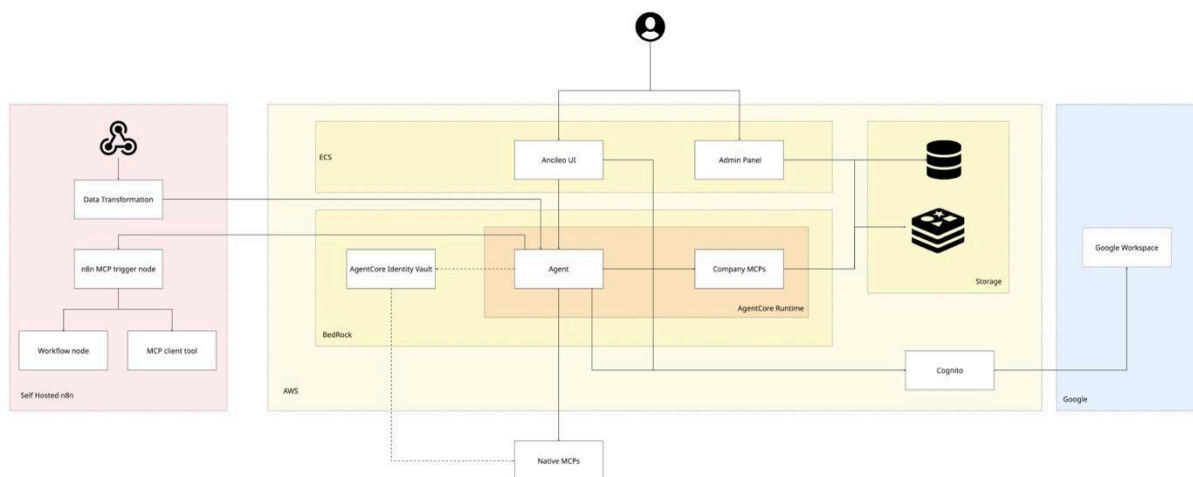
- **AgentCore Runtime Platform:** Migration of Company MCPs from Lambda to AgentCore Runtime with direct user context access and session isolation
- **AgentCore Identity Vault:** Centralized OAuth2 token management for external services with user-specific consent flows and automatic token lifecycle management
- **Custom Angular UI:** Enterprise chat interface deployed on ECS Fargate to replace Claude Desktop dependency with unified Cognito authentication and real-time AgentCore Runtime integration
- **Unified OAuth2 Architecture:** Single authentication flow for both UI access and AgentCore Runtime operations, eliminating multiple login requirements and providing seamless user experience
- **Enhanced Authentication:** Unified Cognito + Google Workspace SSO supporting both UI access and AgentCore Runtime with eliminated API Gateway dependency for Company MCPs
- **Native MCP Integration:** Full external service integration (Atlassian, Google, Slack, GitHub) via AgentCore Identity with user-specific token management
- **n8n Evolution:** Enhanced n8n platform integration with AgentCore Identity for secure external service access within workflows and **ECS RunTask API orchestration for dynamic Cypress container execution**
- **QA Automation Capabilities:** n8n workflows utilize ECS RunTask API to dynamically provision dedicated Fargate tasks for Cypress testing, data processing, and other resource-intensive operations with result aggregation and formatted response delivery
- **Multi-MCP Orchestration:** Complete agent orchestration across Company MCPs (internal), Native MCPs (external), and n8n MCPs (automation) with prompt-first validation and business logic enforcement

Appendix

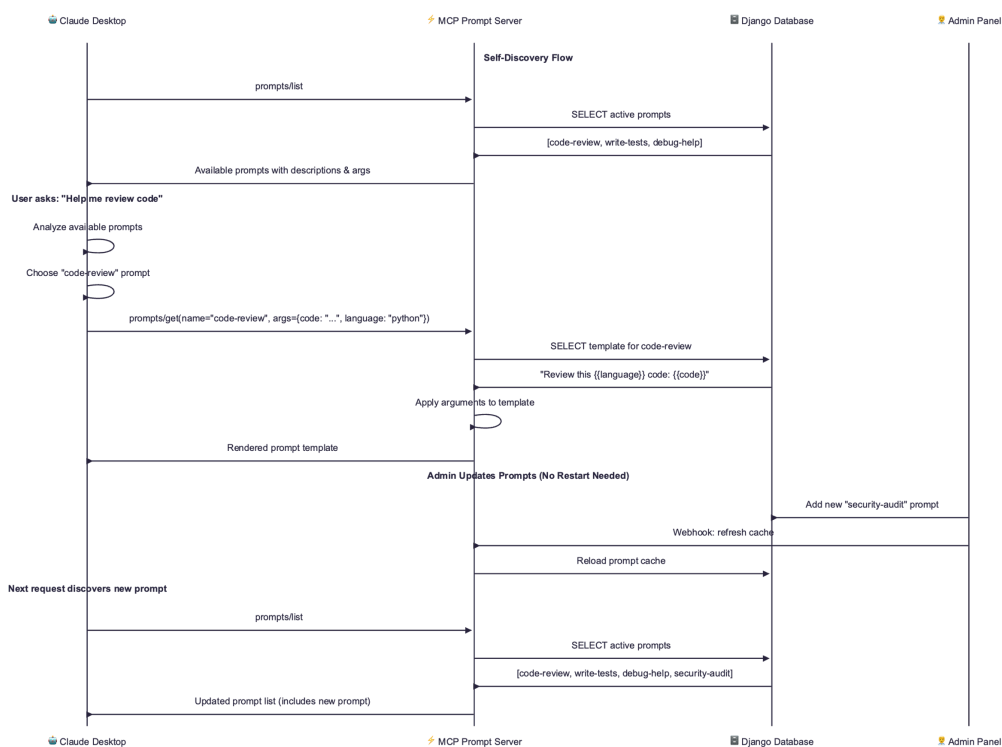
Delivery 1 - Phase 2



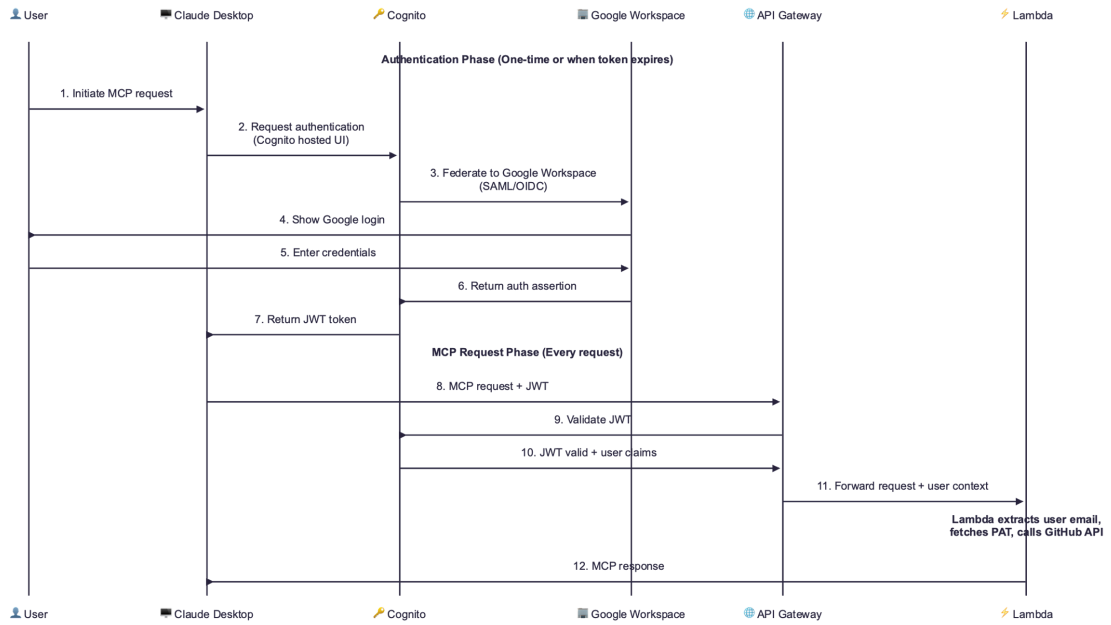
Delivery 2 – Phase 3



Request flow Admin dashboard



Request flow Lambda and Ouath2



Global Flow Delivery 3

